

APN rides along any additive bijection

apn_comp_additive_bijective, read slowly

Abstract

One reusable abstraction.

- `apn_comp_additive_bijective`: if f is APN and σ is an *additive bijection*, then $\sigma \circ f$ is APN.

The Frobenius twist is just the most useful instance. Affine pre/post-composition (EA-equivalence) preserves APN for free, because affine maps are additive bijections.

What APN means

$$\Gamma = (F : \text{Type}, [\text{Field } F], [\text{CharP } F \ 2])$$

$$\text{IsAPN}(f) : \forall a \neq 0, \forall x, y. f(x+a)+f(x) = f(y+a)+f(y) \Rightarrow y = x \vee y = x+a.$$

“Every nonzero direction a has at most two collision partners.”

1 The lemma

```
lemma apn_comp_additive_bijective {F : Type*} [Field F] [CharP F 2]
  {f : F → F} (hf : IsAPN f) σ
  { : F → F} (σh_bij : Function.Bijective σ)
  (σh_add : ∀ x y, σ(x + y) = σx + σy) :
  IsAPN σ( of ) := by
intro a ha x y hxy
convert hf a ha x y _ using 1
exact σh_bij.injective (by aesop)
```

Why it is true (two properties, used once each)

Look at the collision for $\sigma \circ f$:

$$\sigma(f(x+a)) + \sigma(f(x)) = \sigma(f(y+a)) + \sigma(f(y)).$$

1. **Additivity** pulls σ out of each sum:

$$\sigma(f(x+a) + f(x)) = \sigma(f(y+a) + f(y)).$$

2. **Injectivity** (half of bijectivity) cancels σ :

$$f(x+a) + f(x) = f(y+a) + f(y).$$

That is a collision for f . Since f is APN, $y = x$ or $y = x + a$. Done.

$$\frac{\text{IsAPN } f \quad \underbrace{\sigma(u+v) = \sigma u + \sigma v}_{\text{additive}} \quad \underbrace{\text{Injective } \sigma}_{\text{from bijective}}}{\text{IsAPN } (\sigma \circ f)} \text{ APN-TRANSPORT}$$

The two side premises are *exactly* the shapes of `frob_add` (additivity of $x \mapsto x^{2^j}$) and `frob_bijective` (its bijectivity). The lemma is the abstraction; Frobenius is the instance.

2 Instance 1: the Frobenius twist

Take $\sigma(x) = x^{2^j}$. It is additive (`frob_add`) and bijective on a finite field (`frob_bijective`). So multiplying an APN exponent by 2^j keeps it APN:

```
lemma apn_frob_twist {F : Type*} [Field F] [Fintype F] [CharP F 2]
  {d : ℕ} (hd : IsAPN (fun x : F => x ^ d)) (j : ℕ) :
  IsAPN (fun x : F => x ^ (d * 2 ^ j)) := by
  have heq : (fun x : F => x ^ (d * 2 ^ j)) =
    (fun x => x ^ (2 ^ j)) ∘ (fun x => x ^ d) := by
    ext x; simp [pow_mul]
  rw [heq]
  exact apn_comp_additive_bijective hd (frob_bijective_field j) (frob_additive j)
```

The decisive line names the three ingredients in order: `hd` (the APN input), `frob_bijective_field j` (the bijection), `frob_additive j` (the additivity). This is the engine that removes the “ k odd” restriction from Kasami: $x^{d_k} = (x^{d_{n-k}})^{2^{2k}}$.

3 Instance 2: EA-equivalence (for free)

An **affine** map $\sigma(x) = A(x) + c$ with A additive and bijective is an additive bijection up to the constant c — and in the collision both copies of c cancel in char 2. So pre- and post-composing f with affine bijections preserves APN. This is **EA-equivalence**, and it costs nothing beyond the one lemma above.

$$F \xrightarrow[\sim]{\text{affine}} F \xrightarrow{f} F \xrightarrow[\sim]{\text{affine}} F$$

An APN f flanked by affine bijections is still APN.

One-screen summary

ingredient	role
σ additive	pull σ out of the difference
σ injective	cancel σ , recover f 's collision
f APN	≤ 2 partners
$\Rightarrow$	$\sigma \circ f$ APN

additive + bijective $\Rightarrow$ APN transported. Frobenius twist and EA-equivalence are two instances of one lemma.